고급C프로그래밍

05 고급 포인터 응용

free( )의 이해 (1/2)

- **free() 함수 : 사용한 메모리를 반납**

  void free(void ptr);*

  ~~✓ 포인터 변수(ptr)에 널(null) 값을 넣어 준다는 의미~~

  ~~▪ 포인터 변수는 아무것도 가리키지 않으므로, 이 공간을 운영체제에 반납~~

  ✓ 포인터 변수(ptr)의 값을 변경시키지 않음

  ▪ 할당되었던 메모리를 다른 곳에서 사용할 수 있도록 변경

free( )의 이해 (2/2)

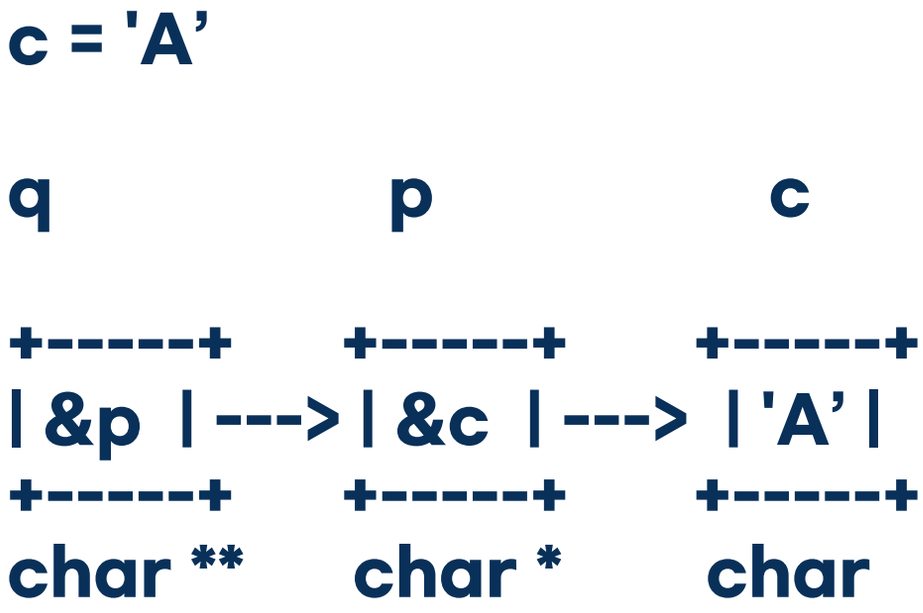
- **고려 사항**
  - ✓ free(ptr);을 호출한 이후에도, ptr을 통해 메모리에 접근 가능
    - (문제점 1) 값을 읽는 경우 ➔ 엉뚱한 값을 읽을 수 있음
    - (문제점 2) 값을 쓰는 경우 ➔ 다른 기능/프로그램의 오작동 유발
  - ✓ 따라서, 해당 문제가 발생하지 않도록 명시적인 처리 필요

```
free(ptr);
ptr = NULL;
```

  - ✓ 왜 C언어에서는 알아서 NULL을 넣어주지 않을까?

포인터 배열의 동적 할당 (1/3)

- **포인터 : 특정 메모리의 주소를 메모리에 저장**
  - ✓ 즉, 특정 메모리의 주소가 저장되는 메모리도 주소가 존재
  - ➔포인터의 포인터
  - ➔ex) chat c = ‘A’; char * p = &c; char **q = &p;



포인터 배열의 동적 할당 (2/3)

- **포인터 배열**
  - ✓ 배열안에 특정 데이터 형식의 값이 아니라, 특정 데이터 형식의 주소를 저장
  - ➔ 결국 새로운 데이터 구조가 아니라 저장하는 값이 다른 배열의 일종
- **N개 크기의 동적 메모리 할당**
  - ✓ 가변 크기의 배열을 생성하는 개념으로 이해

포인터 배열의 동적 할당 (3/3)

- **동적 메모리 할당을 이용한 포인터 배열 생성**
  - ✓ 주소를 저장하는 가변 크기의 배열을 생성
    - 문자형 데이터를 저장하기 위한 n개 크기의 배열 동적 생성

      *char * p = (char *)malloc(sizeof(char) * n)*
    - 문자형 데이터를 저장하는 주소들을 저장하기 위한 n개 크기의 배열 동적 생성

      *char ** q = (char **)malloc(sizeof(char *) * n)*

      ➔ 주소들을 저장하는 메모리 공간의 주소 : 포인터의 포인터로 표현

실습

[실습 1] 대소문자 관계없는 strcmp 구현

- **대소문자에 관계없이 문자열을 비교하는 프로그램**
  - ✓ 사용자에게 문자열 2개를 입력 받음
  - ✓ 문자열 비교 시 대소문자 구분을 하지 않음
  - ✓ malloc() 함수와 포인터 배열을 사용
  - ✓ 지난 시간에 사용한 실습3번의 코드를 응용
  - ✓ 문자열 비교는 my_strcmp( ) 함수를 만들어서 수행

[실습 1] 참고: ASCII 코드표

제어 문자		공백 문자		구두점		숫자		알파벳			
10진	16진	문자	10진	16진	문자	10진	16진	문자	10진	16진	문자
0	0x00	NUL	32	0x20	SP	64	0x40	@	96	0x60	`
1	0x01	SOH	33	0x21	!	65	0x41	A	97	0x61	a
2	0x02	STX	34	0x22	"	66	0x42	B	98	0x62	b
3	0x03	ETX	35	0x23	#	67	0x43	C	99	0x63	c
4	0x04	EOT	36	0x24	$	68	0x44	D	100	0x64	d
5	0x05	ENQ	37	0x25	%	69	0x45	E	101	0x65	e
6	0x06	ACK	38	0x26	&	70	0x46	F	102	0x66	f
7	0x07	BEL	39	0x27	'	71	0x47	G	103	0x67	g
8	0x08	BS	40	0x28	(	72	0x48	H	104	0x68	h
9	0x09	HT	41	0x29	)	73	0x49	I	105	0x69	i
10	0x0A	LF	42	0x2A	*	74	0x4A	J	106	0x6A	j
11	0x0B	VT	43	0x2B	+	75	0x4B	K	107	0x6B	k
12	0x0C	FF	44	0x2C	,	76	0x4C	L	108	0x6C	l
13	0x0D	CR	45	0x2D	-	77	0x4D	M	109	0x6D	m
14	0x0E	SO	46	0x2E	.	78	0x4E	N	110	0x6E	n
15	0x0F	SI	47	0x2F	/	79	0x4F	O	111	0x6F	o
16	0x10	DLE	48	0x30	0	80	0x50	P	112	0x70	p
17	0x11	DC1	49	0x31	1	81	0x51	Q	113	0x71	q
18	0x12	DC2	50	0x32	2	82	0x52	R	114	0x72	r
19	0x13	DC3	51	0x33	3	83	0x53	S	115	0x73	s
20	0x14	DC4	52	0x34	4	84	0x54	T	116	0x74	t
21	0x15	NAK	53	0x35	5	85	0x55	U	117	0x75	u
22	0x16	SYN	54	0x36	6	86	0x56	V	118	0x76	v
23	0x17	ETB	55	0x37	7	87	0x57	W	119	0x77	w
24	0x18	CAN	56	0x38	8	88	0x58	X	120	0x78	x
25	0x19	EM	57	0x39	9	89	0x59	Y	121	0x79	y
26	0x1A	SUB	58	0x3A	:	90	0x5A	Z	122	0x7A	z
27	0x1B	ESC	59	0x3B	;	91	0x5B	[	123	0x7B	{
28	0x1C	FS	60	0x3C	<	92	0x5C	₩	124	0x7C	|
29	0x1D	GS	61	0x3D	=	93	0x5D	]	125	0x7D	}
30	0x1E	RS	62	0x3E	>	94	0x5E	^	126	0x7E	~
31	0x1F	US	63	0x3F	?	95	0x5F	_	127	0x7F	DEL

[실습 2] 문자열 소문자-대문자 변환기

- **사용자로부터 N개의 문자열을 입력 받고, 해당 문자열을 모두 대문자와 소문자로 변환하여 출력**
  - ✓ 문자열에 있는 각 문자를 대문자/소문자로 변환하여 출력하는 프로그램(대문자 ➔ 소문자, 소문자 ➔ 대문자)
  - ✓ malloc() 함수와 포인터 배열의 동적 할당을 사용
  - ✓ 변환 기능은 서브 함수를 정의해서 구현

[실습 3] 문자열 세로로 읽기

- **사용자로부터 N개의 문자열 입력받고, 해당 문자열을 세로로 출력**
  - ✓ malloc()/realloc() 함수와 포인터 배열의 동적 할당을 사용
  - ✓ 세로로 읽을 시 특정 문자열에 해당 자리의 문자가 없으면, 다음 문자열의 문자를 읽음
  - ✓ 변환 기능은 서브 함수를 정의해서 구현

ABCDE
abcde
01234
FGHIJ
fghij

AABCD
afz
09121
8EWg6
P5h3kx

Aa0FfBb1GgCc2HhDd3IiEe4Jj

Aa09PAf9E5Bz1WhC2g3D16kx

[실습 4] 로마숫자 변환기

- **사용자에게 1000 이하의 숫자를 입력받고, 이를 로마 숫자로 변환하는 프로그램 작성**
  - ✓ 기본적으로 큰숫자에서 작은 숫자순으로 작성(왼쪽 ➔ 오른쪽)
  - ✓ 작은 숫자가 큰 숫자 왼쪽에 오는 경우
  - ➔ IV : 4, IX : 9, XL : 40, XC : 90, CD : 400, CM : 900
  - ✓ V, L, D는 한번만 사용할 수 있고, I, X, C, M은 연속해서 3번까지 사용할 수 있음
  - ✓ 가능한 적은 개수의 로마숫자들로 표현
  - ✓ malloc()/realloc() 함수 사용
  - ✓ 변환 기능은 서브 함수를 정의해서 구현

[실습 4] 로마숫자 변환기

기호	I	V	X	L	C	D	M
10진수	1	5	10	50	100	500	1000

<출력 순서>
입력 수 = 수식 = 로마숫자, 문자열 길이

<출력 예시>
553 = 500 + 50 + 3 = DLIII, 5
940 = 900 + 40 = CMXL, 4
235 = 200 + 30 + 5 = CCXXXV, 6
493 = 400 + 90 + 3 = CDXCIII, 7

Q & A